

Section 2

2

Pilih semua pernyataan yang BENAR.

```
String a = "hello";  
a = a + " world";
```

- ☐ Variabel a diubah secara langsung (in-place) menjadi "hello world"
- ☒ String di Java bersifat immutable objek "hello" yang lama tidak berubah
- ☒ Sebuah objek String baru diciptakan, dan objek lama dibuang untuk dipungut oleh garbage collector
- ☒ Untuk operasi yang banyak melibatkan perubahan string, sebaiknya menggunakan StringBuilder

3

Pilih semua pernyataan yang BENAR.

```
StringBuilder sb = new StringBuilder("mataku");  
sb.setCharAt(4, 'm');  
sb.append(" biru");  
String s = sb.toString();
```

- ☒ Nilai s adalah "matamu biru"
- ☐ `StringBuilder sb = "hello";` adalah kode yang valid
- ☒ `StringBuilder` bersifat mutable, perubahan dilakukan tanpa menciptakan objek baru
- ☐ `StringBuilder` mendukung method chaining karena method-nya me-return this (fluent interface)

Pilih semua pernyataan yang BENAR.

```
class ReportService {  
    public void generatePDF(Data data) { /* generate PDF */ }  
    public void generateCSV(Data data) { /* generate CSV */ }  
    public void sendEmail(String to, String body) { /* send email */ }  
    public void logToDatabase(String msg) { /* log to DB */ }  
}
```

- ☐ Desain ini sudah baik karena semua method berkaitan dengan "report"
- ☐ Desain ini melanggar Single Responsibility Principle karena class memiliki lebih dari satu alasan untuk berubah (reason to change)
- ☒ Single Responsibility Principle menyatakan bahwa sebuah class hanya boleh memiliki satu method
- ☐ Sebaiknya tanggung jawab pengiriman email dan logging dipisahkan ke class tersendiri agar setiap class merepresentasikan satu tanggung jawab konseptual
- ☒ Sebaiknya tanggung jawab pengiriman email dan logging dipisahkan ke class tersendiri agar setiap class merepresentasikan satu tanggung jawab konseptual

Pilih semua pernyataan yang BENAR.

Desain A :

```
class GraphicEditor {  
    public void drawShape(Shape s) {  
        if (s.type == 1) drawRectangle();  
        else if (s.type == 2) drawCircle();  
    }  
}
```

Desain B :

```
class GraphicEditor {  
    public void drawShape(Shape s) {  
        s.draw();  
    }  
}  
  
abstract class Shape {  
    abstract void draw();  
}  
  
class Rectangle extends Shape {  
    void draw() { /* draw rectangle */ }  
}  
  
class Circle extends Shape {  
    void draw() { /* draw circle */ }  
}
```



Desain A sudah memenuhi Open/Closed Principle



Pada Desain A, menambahkan shape baru mengharuskan modifikasi class GraphicEditor, sehingga melanggar Open/Closed Principle



Desain B memenuhi Open/Closed Principle karena sistem dapat di-extend (menambah shape baru) tanpa mengubah bagian yang stabil (GraphicEditor)



Open/Closed Principle menyatakan bahwa entitas harus open for extension, but closed for modification

Pilih semua pernyataan yang BENAR.

Bad Example :

```
class Worker {  
    public void work() { /* some work */ }  
}  
  
class Manager {  
    Worker worker;  
    public void setWorker(Worker w) { worker = w; }  
    public void manage() { worker.work(); }  
}
```

Good Example :

```
interface IWorker {  
    public void work();  
}  
  
class Worker implements IWorker {  
    public void work() { /* some work */ }  
}  
  
class SuperWorker implements IWorker {  
    public void work() { /* some heavy work */ }  
}  
  
class Manager {  
    IWorker worker;  
    public void setWorker(IWorker w) { worker = w; }  
    public void manage() { worker.work(); }  
}
```

Bad example melanggar Dependency Inversion

- ✓ Principle karena class Manager (high-level) bergantung langsung pada class Worker (low-level)

Good example melanggar Liskov Substitution

- ☐ Principle karena SuperWorker tidak bisa menggantikan Worker

Pada Good example, Manager bergantung pada abstraksi (IWorker), bukan pada implementasi konkret



Dependency Inversion Principle menyatakan bahwa modul high-level tidak boleh bergantung pada modul low-level, melainkan keduanya bergantung pada abstraksi



Pilih semua pernyataan yang BENAR.

```
abstract class Bangun {  
    void test() {  
        System.out.println("Luas" + getLuas());  
    }  
    abstract double getLuas();  
}  
  
class Lingkaran extends Bangun {  
    double r;  
  
    Lingkaran(double r) { this.r = r; }  
  
    @Override  
    double getLuas() { return Math.PI * r * r; }  
}
```

Kemudian :

```
Bangun b = new Bangun();  
Bangun c = new Lingkaran(5);  
c.getLuas();
```

- ☒ Bangun b = new Bangun(); akan menghasilkan compile error karena kelas abstrak tidak dapat diinstansiasi
- ☐ Bangun c = new Lingkaran(5); akan menghasilkan compile error karena tipe variabel dan objek berbeda
- ☒ Lingkaran wajib mengimplementasikan method getLuas() karena method tersebut bersifat abstract di superclass
- ☒ Jika kelas turunan tidak mengimplementasikan semua method abstrak, maka kelas turunan

Jika kelas turunan tidak mengimplementasikan

✓ semua method abstrak, maka kelas turunan tersebut harus dideklarasikan abstract juga

8

Asumsikan Lingkaran dan Segitiga keduanya extends Bangun dan masing-masing meng-override method getLuas(). Pilih semua pernyataan yang BENAR.

```
Lingkaran l = new Lingkaran(10);  
Segitiga s = new Segitiga(1, 2, 3);
```

```
Bangun b = l;  
System.out.println(b.getLuas());
```

```
b = s;  
System.out.println(b.getLuas());
```

Asumsikan Lingkaran dan Segitiga keduanya extends Bangun dan masing-masing meng-override method getLuas(). Pilih semua pernyataan yang BENAR.

```
Lingkaran l = new Lingkaran(10);  
Segitiga s = new Segitiga(1, 2, 3);  
  
Bangun b = l;  
System.out.println(b.getLuas());  
  
b = s;  
System.out.println(b.getLuas());
```

- ☒ Pemanggilan pertama b.getLuas() akan mencetak luas lingkaran, bukan luas bangun
- ☒ Java menganut dynamic polymorphism, sifat sebuah objek ditentukan berdasarkan reference yang ditunjuknya, bukan tipe pada saat deklarasi
- ☐ Karena b bertipe Bangun, maka b.getLuas() selalu memanggil method getLuas() milik kelas Bangun
- ☒ Jika sebuah method menerima parameter bertipe Bangun, ia dapat di-supply dengan Lingkaran karena hubungan is-a

Pilih semua pernyataan yang BENAR.

```
class Person {  
    String name;  
    Person(String name) {  
        this.name = name;  
    }  
}  
  
class Student extends Person {  
    String studentId;  
    Student(String name, String studentId) {  
        super(name);  
        this.studentId = studentId;  
    }  
}
```

- ☐ Pemanggilan `super(name)` boleh ditempatkan di mana saja dalam konstruktor `Student`
- ☒ `super(name)` harus merupakan statement pertama dalam konstruktor `Student`
- Jika `super(...)` tidak dipanggil secara eksplisit,
- ☒ Java akan otomatis memanggil konstruktor default (tanpa parameter) milik parent
- ☐ Terjadi constructor chaining: konstruktor semua kelas "leluhur" akan dipanggil

Pilih semua pernyataan yang BENAR.

```
final class String { /* ... */ }  
class MyString extends String { /* ... */ }
```

class MyString extends String akan

- ☒ menghasilkan compile error karena kelas String ditandai final dan tidak dapat diturunkan
- ☐ Keyword final pada kelas hanya mencegah method di dalamnya untuk di-override, tapi kelas masih bisa diturunkan
- ☐ Sebuah method yang ditandai final tidak dapat di-override oleh subclass
- ☒ Contoh kelas final di Java antara lain System dan String

Pilih semua pernyataan yang BENAR.

- ☒ C++ mendukung multiple inheritance pada class (contoh: `class C : public A, public B`), sedangkan Java melarang multiple inheritance pada class
- ☒ Java menghindari multiple inheritance pada class untuk mencegah diamond problem yang sulit diprediksi
- ☐ Di Java, programmer harus mengelola memori secara manual menggunakan `new/delete` seperti di C++
- ☒ Di Java, Garbage Collector membebaskan memori secara otomatis, sementara di C++ programmer bertanggung jawab penuh (RAII / `new/delete`)

Pilih semua pernyataan yang BENAR

- ☒ Di C++ bisa men-throw objek apapun (termasuk int atau string literal), sedangkan di Java yang di-throw harus turunan dari Throwable
- ☐ C++ memiliki keyword finally yang setara dengan Java
- ☒ Java, checked exception harus ditangani (try-catch) atau dideklarasikan (throws), sedangkan C++ tidak memiliki konsep ini
- ☒ Di C++, main() adalah fungsi bebas di luar kelas, sedangkan di Java main() adalah method static di dalam kelas

Pilih semua pernyataan yang BENAR.

```
public interface Valuable {  
    int value();  
    default String label() { return "Item worth " + value(); }  
}  
  
public class ValuableHouse extends House implements Valuable {  
    private double locationIndex;  
  
    public ValuableHouse(int bedrooms, double locationIndex) {  
        super(bedrooms);  
        this.locationIndex = locationIndex;  
    }  
  
    @Override  
    public int value() {  
        return (int) (getNoOfBedrooms() * 50000 * locationIndex);  
    }  
}
```

ValuableHouse wajib mengimplementasikan



method value() karena merupakan abstract method di interface Valuable



Valuable[] arr = new Valuable[5]; adalah deklarasi yang valid



ValuableHouse wajib meng-override method label() agar bisa dikompilasi



ValuableHouse bersifat polymorphic, ia bisa diperlakukan sebagai House maupun Valuable

Pilih semua pernyataan yang BENAR.

```
interface Flyable {  
    void fly();  
    default String mode() { return "airborne"; }  
}  
  
interface Swimmable {  
    void swim();  
    default String mode() { return "aquatic"; }  
}  
  
class Duck implements Flyable, Swimmable {  
    public void fly() { System.out.println("Flap wings"); }  
    public void swim() { System.out.println("Paddle"); }  
}
```



Kode di atas dapat dikompilasi tanpa error

Compiler akan error karena Duck mewarisi dua



default method `mode()` yang konflik dan tidak melakukan override



Konflik dapat diselesaikan dengan menambahkan override method `mode()` di class Duck

Menambahkan `extends Animal` setelah Duck



akan menyebabkan error karena Java tidak mengizinkan `extends` dan `implements` sekaligus

Pilih semua pernyataan yang BENAR.

```
public interface Measurable {  
    double getArea();  
    double getPerimeter();  
}  
  
public abstract class Shape {  
    protected String color;  
    public abstract void draw();  
}  
  
public class Circle extends Shape implements Measurable {  
    private double radius;  
  
    public Circle(double radius) { this.radius = radius; }  
    public void draw() { System.out.println("Drawing circle"); }  
    public double getArea() { return Math.PI * radius * radius; }  
    public double getPerimeter() { return 2 * Math.PI * radius; }  
}
```

☐ Interface Measurable boleh memiliki field private double size;

Circle harus mengimplementasikan draw(), get-

☒ Area(), dan getPerimeter() agar tidak menjadi abstract class

Measurable m = new Circle(5); adalah per-

☐ nyataan yang valid, dan m.getArea() dapat dipanggil

Interface digunakan di sini karena Measurable

☒ hanya memberi kontrak tanpa implementasi, dan bisa dipakai oleh kelas-kelas yang tidak saling berhubungan


```
public class Box<T> {  
    private T t;  
  
    public void put(T t) {  
        this.t = t;  
    }  
  
    public T get() {  
        return t;  
    }  
}
```

Kemudian digunakan sebagai berikut:

```
Box<Integer> intBox = new Box<>();  
intBox.put(10);
```

- ☒ T merupakan parameter tipe generik
- ☒ intBox.put("10") akan menghasilkan compile error
- ☒ Method get() mengembalikan Object
- ☐ Generics mengurangi kebutuhan explicit casting


```
public static <U> void fillBoxes(U u, List<Box<U>> boxes)
```

dan pemanggilan:

```
Crayon red = new Crayon("red");  
List<Box<Crayon>> crayonBoxes = new ArrayList<>();  
  
Box.fillBoxes(red, crayonBoxes);
```

- ☒ Compiler melakukan type inference terhadap tipe U
- ☒ Pada pemanggilan di atas, U akan diinfer sebagai Crayon
- ☐ Generic method hanya dapat digunakan di dalam generic class
- ☒ Penulisan Box.<Crayon>fillBoxes(...) juga valid

```
public void doSomethingToBoxOfNumber (Box<Number> b) {  
    /*...*/  
}
```

```
Box<Integer> intBox = new Box<>();  
Box<Double> dblBox = new Box<>();
```

- ✓ doSomethingToBoxOfNumber(intBox) valid karena Integer adalah Number
- ✓ Box<Integer> merupakan subtype dari Box<Number>
- ✓ Box<Double> dapat langsung di-assign ke Box<Number>
- ✓ Integer merupakan subtype dari Number

Perhatikan pernyataan berikut. Pilih semua pernyataan yang benar.

- ☒ `List<String>` dapat langsung dianggap sebagai `List<Object>` karena `String` adalah turunan dari `Object`.
- ☒ `List<?>` dapat digunakan untuk merepresentasikan list dengan tipe elemen yang tidak diketahui.
- ☐ `List<Object>` adalah leluhur dari semua jenis `List<T>`.
- ☒ `List<?>` dapat menerima referensi dari `List<String>`.

Pilih semua operasi yang valid secara kompilasi.

```
List<?> lu = new ArrayList<String>();
```

- ☒ lu.add("hello");
- ☐ lu.add(new Object());
- ☒ Object x = lu.get(0);
- ☒ String s = lu.get(0);

```
void feedAnimals(List<? extends Animal> l) {  
    for (Animal a : l) {  
        a.feedMe();  
    }  
}
```

- ☒ Method tersebut dapat menerima List<Animal>.
- ☒ Method tersebut dapat menerima List<Lion>.

Method tersebut dapat menerima

- ☒ List<Butterfly>, selama Butterfly turunan dari Animal.
- ☐ Di dalam method, aman melakukan l.add(new Lion()).
- ☒ Di dalam method, aman membaca elemen sebagai Animal.

Java Collection Framework disebut sebagai unified architecture untuk merepresentasikan dan memanipulasi collection. Komponen apa saja yang termasuk di dalamnya?

- ☒ Interfaces
- ☒ Implementations
- ☒ Algorithms
- ☐ Compiler

23

Manakah pernyataan yang benar mengenai core collection interfaces di Java?

- ☒ Semua core collection interfaces bersifat generic
- ☒ Collection<E> menunjukkan bahwa collection menyimpan elemen bertipe E
- ☐ Map adalah turunan langsung dari Collection
- ☒ Set, List, Queue, dan Deque berada di bawah Collection

Pilih pernyataan yang BENAR.

```
myList
    .stream()
    .filter(s -> s.startsWith("c"))
    .map(String::toUpperCase)
    .sorted()
    .forEach(System.out::println);
```

- ☒ filter digunakan untuk memilih elemen tertentu dari stream
- ☒ map digunakan untuk melakukan transformasi elemen
- ☒ sorted digunakan untuk mengurutkan elemen
- ☐ forEach menghasilkan stream baru dengan tipe berbeda
- ☒ Stream API memungkinkan pemrosesan collection secara fluent

25

Dalam Java, manakah pernyataan yang benar tentang exception?

- ☒ Exception adalah event yang terjadi saat eksekusi program dan mengganggu alur normal instruksi.
- ☒ Exception object berisi informasi tentang error, termasuk tipe dan state program ketika error terjadi.
- ☐ Exception hanya terjadi saat compile time.
- ☒ Throwing an exception berarti membuat exception object dan melemparkannya ke runtime system.

26

Perhatikan struktur exception handling berikut. Pernyataan mana yang benar?

- ☒ try berisi kode yang mungkin memunculkan exception.
- ☒ catch menangani exception berdasarkan tipe yang sesuai.
- ☐ Satu try hanya boleh memiliki satu catch.
- ☒ finally tetap dijalankan walaupun terjadi return atau throw di dalam try atau catch.

27

Manakah pernyataan yang benar mengenai try-with-resources?

- ☒ Diperkenalkan mulai Java 7.
- ☒ Resource akan otomatis di-close() setelah blok selesai.
- ☒ Resource harus mengimplementasikan AutoCloseable.
- ☐ try-with-resources selalu menghilangkan kebutuhan catch.

Dalam konteks Java multithreading, pernyataan mana saja yang benar?

- ☒ Thread adalah alur eksekusi di dalam sebuah process.
- ☐ Beberapa thread dalam satu process dapat berbagi resource process yang sama.
- ☒ Multithreading selalu berarti semua thread berjalan benar-benar paralel pada core berbeda.
- ☒ Multithreading dapat membantu aplikasi tetap responsif ketika ada proses yang lama.

Perhatikan pernyataan berikut tentang penggunaan Thread dan Runnable di Java. Pernyataan mana saja yang benar?

- ☒ Jika sebuah class sudah extends class lain, maka class tersebut tidak dapat lagi extends Thread.
- ☐ Class yang implements Runnable dapat langsung dipanggil method start().
- ☒ Runnable mendefinisikan pekerjaan yang akan dijalankan, sedangkan Thread dapat digunakan untuk menjalankan pekerjaan tersebut.
- ☒ Method run() berisi kode yang akan dijalankan oleh thread.

30

Pada kasus PizzaBox, terdapat producer yang menaruh pizza dan consumer yang mengambil pizza. Pernyataan mana saja yang benar?

☐ Variabel available digunakan untuk menandakan apakah pizza sedang tersedia atau tidak.

☒ Busy waiting seperti while (!available); harus dihindari karena membuang waktu CPU.

☒ wait() membuat thread menunggu dan dapat melepaskan lock sementara agar thread lain dapat mengubah kondisi.

☒ notify() membangunkan semua thread yang sedang menunggu.

Back

Submit